



RAJALAKSHMI ENGINEERING COLLEGE
(Autonomous Institution, Affiliated to Anna University, Chennai)
Thandalam, Chennai - 602 105

**GE19612 - PROFESSIONAL READINESS FOR INNOVATION,
EMPLOYABILITY AND ENTREPRENEURSHIP PROJECT
REPORT**

Submitted by

KARTHIKHA SRE M (2116230701143)
SARANYA M (2116230701510)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE
CHENNAI

MAY 2026

RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project titled "Explainable AI-Based Heart Disease Prediction System Using Ensemble Learning" is the bonafide work of KARTHIKHA SRE M (2116230701143) and SARANYA M (2116230701510), who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. _____, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

Professor

Department of Computer Science and Engineering,

Rajalakshmi Engineering College, Chennai - 602 105.

SIGNATURE

Dr. _____, M.E., Ph.D.,

SUPERVISOR

Assistant Professor

Department of Computer Science and Engineering,

Rajalakshmi Engineering College, Chennai - 602 105.

Submitted to Mini Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Cardiovascular disease remains one of the foremost causes of mortality worldwide, making timely and accurate clinical prediction an urgent public health imperative. This project presents an Explainable AI-Based Heart Disease Prediction System that combines the predictive strength of ensemble machine learning with the interpretive power of modern explainability frameworks. The system employs a Stacking Classifier integrating Logistic Regression, Random Forest, and K-Nearest Neighbors as base learners, trained on the Cleveland Heart Disease dataset and preprocessed using a robust pipeline incorporating StandardScaler and OneHotEncoder through a ColumnTransformer. Developed as a full-stack web application using React.js for the frontend and FastAPI (Python) for the backend, the platform accepts thirteen clinical parameters, produces a risk prediction with a calibrated risk score, and classifies the output into Low, Medium, or High risk categories. To bridge the gap between algorithmic prediction and clinical understanding, the system integrates LIME (Local Interpretable Model-Agnostic Explanations) and SHAP (SHapley Additive exPlanations) to generate local and global feature importance visualizations, respectively. Natural language clinical explanations and personalized medical recommendations are also generated for each patient case. The system achieved a prediction accuracy exceeding 87 percent on the test set. By offering a transparent, doctor-friendly, and accessible diagnostic tool, this work addresses critical limitations of black-box models in clinical settings and demonstrates how explainable AI can meaningfully augment clinical decision-making in cardiology.

ACKNOWLEDGMENT

We express our deepest gratitude to the Almighty for His blessings throughout this endeavor. We extend our sincere thanks to the management of Rajalakshmi Engineering College for providing excellent infrastructure and resources to support our academic and research activities.

We are grateful to our Principal, Dr. S.N. MURUGESAN, M.E., Ph.D., for his constant encouragement and leadership. We owe special thanks to Dr. P. KUMAR, M.E., Ph.D., Professor and Head of the Department of Computer Science and Engineering, for his guidance and support throughout the project.

We sincerely thank our project supervisor for his invaluable mentorship, technical guidance, and patience during the development of this work. We also acknowledge the support of our faculty colleagues and project coordinator for their constructive feedback during the review process.

Finally, we thank our families and friends for their moral support and encouragement that motivated us to complete this project successfully.

KARTHIKHA SRE M	2116230701143
SARANYA M	2116230701510

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	ACKNOWLEDGMENT	ii
	TABLE OF CONTENTS	iii
	LIST OF TABLES	iv
	LIST OF FIGURES	v
	LIST OF ABBREVIATIONS	vi
1	INTRODUCTION	1
	1.1 Background and Motivation	1
	1.2 Limitations of Traditional Diagnosis	1
	1.3 Need for AI-Based Systems	2
	1.4 Introduction to Explainable AI	2
	1.5 Objectives	3
2	LITERATURE SURVEY	4
3	SYSTEM OVERVIEW	7
4	SYSTEM ARCHITECTURE	9
5	METHODOLOGY / APPROACH	11
6	IMPLEMENTATION DETAILS	15
7	RESULTS AND DISCUSSION	20
8	CONCLUSION	24
9	FUTURE ENHANCEMENTS	25
	REFERENCES	27

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
5.1	Cleveland Heart Disease Dataset Features	12
5.2	Model Performance Comparison	21
7.1	Risk Classification Output Examples	21

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	System Architecture Diagram	10
6.1	Patient Input Form (Frontend UI)	17
7.1	Prediction Results with Risk Classification	21
7.2	LIME Feature Contribution Chart	22
7.3	SHAP Feature Contribution Chart	23

LIST OF ABBREVIATIONS

S. No.	ABBREVIATION	EXPANSION
1	AI	Artificial Intelligence
2	ML	Machine Learning
3	XAI	Explainable Artificial Intelligence
4	LIME	Local Interpretable Model-Agnostic Explanations
5	SHAP	SHapley Additive exPlanations
6	RF	Random Forest
7	LR	Logistic Regression
8	KNN	K-Nearest Neighbors
9	API	Application Programming Interface
10	REST	Representational State Transfer
11	ECG	Electrocardiogram

12	CSV	Comma-Separated Values
13	UI	User Interface
14	JSON	JavaScript Object Notation
15	HTTP	Hypertext Transfer Protocol

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Heart disease represents a spectrum of cardiovascular conditions — including coronary artery disease, arrhythmia, heart failure, and valvular disease — that collectively constitute the leading cause of death globally, accounting for approximately 17.9 million deaths per year according to the World Health Organization. In India, cardiovascular disease contributes to nearly 28 percent of all deaths, with urban populations increasingly affected due to sedentary lifestyles, dietary changes, and heightened stress levels. The economic burden of cardiac events extends beyond mortality; survivors often face prolonged hospitalization, disability, and diminished quality of life. Given this overwhelming epidemiological context, the ability to accurately predict individual risk at an early stage becomes not merely clinically valuable but socially essential.

Clinical prediction of heart disease has traditionally relied on a combination of physical examination findings, laboratory investigations, and expert judgment. While tools such as the Framingham Risk Score and its derivatives have provided structured frameworks for risk stratification, their applicability is limited by their reliance on a small number of variables and their inability to capture complex, non-linear interactions between risk factors. Moreover, access to experienced cardiologists remains unevenly distributed, particularly in semi-urban and rural settings across India. In such environments, a data-driven, AI-assisted screening tool capable of operating with basic clinical parameters and producing interpretable outputs can serve as a meaningful bridge between population-level health data and actionable clinical decisions.

1.2 Limitations of Traditional Diagnosis

Traditional diagnostic approaches for heart disease carry several systemic limitations that restrict their efficacy in contemporary healthcare settings. First, they are inherently retrospective and symptom-driven; patients typically seek medical evaluation after experiencing warning signs such as chest pain, dyspnea, or palpitations, by which point significant cardiac damage may have already occurred. Second, conventional methods depend heavily on clinical intuition accumulated through years of practice, making them difficult to standardize across different healthcare providers and institutions. Third, diagnostic tools such as echocardiography, stress testing, and coronary angiography, while accurate, are expensive, time-consuming, and require specialized infrastructure, rendering them inaccessible to a large segment of the population. Finally, human decision-making in clinical contexts

is inherently subject to cognitive biases and fatigue-related errors, especially in high-volume settings where physicians may evaluate dozens of patients in a single session.

1.3 Need for AI-Based Systems

Artificial Intelligence, and in particular machine learning, has demonstrated remarkable capability in extracting meaningful patterns from high-dimensional clinical datasets. Unlike traditional statistical models, machine learning algorithms can simultaneously process a large number of variables and identify complex, non-linear relationships that escape conventional analytical techniques. Studies have shown that ensemble learning methods — which aggregate the outputs of multiple base classifiers — consistently outperform single models in terms of predictive accuracy and generalization, particularly in imbalanced clinical datasets. The availability of structured heart disease datasets, combined with advances in computational resources and open-source libraries, has created a favorable environment for developing AI-powered diagnostic support tools that can be deployed in a wide range of clinical contexts. Such systems do not intend to replace medical professionals but to augment their capabilities, reduce diagnostic variability, and enable earlier, more consistent identification of at-risk individuals.

1.4 Introduction to Explainable AI

Despite the demonstrated predictive power of machine learning models in healthcare applications, their widespread clinical adoption has been impeded by a fundamental limitation: most high-performing models, particularly ensemble methods and deep neural networks, operate as black boxes whose internal decision-making processes are opaque to end users. A physician relying on such a system cannot understand why a particular prediction was made, which clinical features drove the output, or how confident the model is in its assessment. This lack of transparency raises significant concerns related to accountability, patient safety, and regulatory compliance, particularly in a domain as consequential as cardiac care.

Explainable Artificial Intelligence (XAI) addresses this challenge by providing post-hoc or inherently interpretable frameworks that articulate the reasoning behind model predictions. Two of the most widely adopted XAI techniques are LIME and SHAP. LIME, developed by Ribeiro et al., operates by fitting a locally interpretable surrogate model in the neighborhood of a specific prediction, thereby revealing which features contributed most to that particular output. SHAP, grounded in cooperative game theory, computes Shapley values that fairly allocate the prediction's deviation from the mean across all contributing features, providing both local and global interpretability. By integrating these two complementary explainability frameworks into a clinically-facing application, this project aims to

close the transparency gap between predictive algorithms and clinical users, thereby enhancing trust, usability, and ultimately patient safety.

1.5 Objectives

The primary objectives of this project are as follows. First, to develop a machine learning pipeline utilizing a Stacking Classifier that combines Logistic Regression, Random Forest, and K-Nearest Neighbors to predict the presence of heart disease from clinical parameters with high accuracy. Second, to integrate LIME and SHAP as post-hoc explainability mechanisms that provide both local (per-patient) and global (model-level) feature attribution. Third, to construct a full-stack web application using React.js and FastAPI that offers a clinical user interface through which patient data can be entered, predictions can be visualized, and explanations can be reviewed in a meaningful format. Fourth, to provide risk stratification into Low, Medium, and High categories along with a quantitative risk score and personalized medical recommendations. Fifth, to ensure that the system is accessible, intuitive, and designed with the needs of non-technical clinical users in mind.

CHAPTER 2

LITERATURE SURVEY

The intersection of machine learning and cardiovascular medicine has attracted considerable research attention over the past decade, resulting in a rich body of literature on AI-based heart disease prediction systems. A thorough review of existing works reveals both the progress achieved and the persistent challenges that motivate the current study.

2.1 Early Machine Learning Approaches

Detrano et al. (1989) were among the first to systematically apply probabilistic methods to coronary artery disease prediction, utilizing the Cleveland Heart Disease dataset that subsequently became the benchmark dataset for the field. Their logistic regression model established a foundational baseline but was limited in its ability to capture non-linear feature interactions. Subsequent work by Ster and Dobnikar (1996) applied neural networks to the same dataset and demonstrated superior discriminative ability, particularly for cases with complex co-morbidities. However, these early systems lacked any mechanism for explaining individual predictions, limiting their clinical utility.

Palaniappan and Latif (2008) conducted a comparative study of three machine learning classifiers — Decision Tree, Naive Bayes, and Neural Network — on a heart disease dataset. Their findings indicated that Naive Bayes achieved the highest accuracy among the three, but all models were constrained by limited feature sets and an absence of preprocessing standardization. The study highlighted that model selection alone is insufficient without attention to data quality, feature engineering, and appropriate evaluation protocols. Similar comparative analyses were conducted by Choudhury (2017) and Bashir et al. (2019), who extended the scope to include Support Vector Machines and Random Forests, respectively, confirming that ensemble-based methods consistently deliver superior performance in medical classification tasks.

2.2 Ensemble Learning for Heart Disease

The application of ensemble learning to heart disease classification has been extensively explored. Pouriyeh et al. (2017) demonstrated that boosting and bagging ensemble techniques significantly improved prediction accuracy compared to standalone classifiers in a cardiac dataset, reducing both false positive and false negative rates. Golande and Pavankumar (2019) proposed a hybrid ensemble approach combining majority voting across five classifiers — SVM, K-NN, Decision Tree, Naive Bayes, and Random Forest — and reported an accuracy of 86 percent on the UCI Heart Disease dataset.

While these studies affirmed the efficacy of ensemble strategies, they did not incorporate any explainability mechanisms, making them unsuitable for direct clinical deployment.

The concept of stacking, or stacked generalization, as introduced by Wolpert (1992), offers a more sophisticated approach to ensemble learning in which a meta-learner is trained on the outputs of multiple base learners. Feng et al. (2019) applied this technique to cardiac risk prediction and observed that stacking consistently outperformed bagging and boosting under comparable conditions, a finding corroborated by subsequent studies in medical diagnostics. However, the interpretability of stacked classifiers is inherently more challenging than simpler models, underscoring the necessity of coupling stacking with explainability methods.

2.3 Explainable AI in Healthcare

The application of explainable AI to healthcare has emerged as a critical research frontier in response to the clinical accountability demands that opaque models cannot satisfy. Ribeiro et al. (2016) introduced LIME and demonstrated its ability to generate locally faithful explanations for any black-box classifier through surrogate modeling. Their experiments on medical datasets showed that LIME explanations were both comprehensible to non-expert users and aligned with clinical domain knowledge.

Lundberg and Lee (2017) introduced SHAP values, which provide a theoretically grounded and computationally efficient approach to feature attribution based on the Shapley values of cooperative game theory. Unlike LIME, which provides local approximations, SHAP can generate globally consistent explanations that are stable across the full dataset. Subsequent work by Lundberg et al. (2020) specifically applied TreeSHAP to clinical prediction models and demonstrated significant improvements in clinician trust and model adoption. Hsieh et al. (2021) applied SHAP to a cardiovascular risk model and found that SHAP-generated insights closely mirrored established clinical knowledge about risk factors, including the primacy of age, cholesterol, and maximum heart rate in cardiac risk stratification.

2.4 Limitations of Existing Systems

A systematic examination of existing heart disease prediction systems reveals several recurring limitations. First, most published systems are developed as standalone scripts or Jupyter notebooks rather than deployable web applications, limiting their real-world clinical utility. Second, many studies report accuracy on test splits without providing confidence intervals, robustness analyses, or calibration metrics, raising questions about external validity. Third, single-model systems — particularly those based on deep learning — tend to overfit on small clinical datasets and sacrifice

interpretability for marginal gains in accuracy. Fourth, while some recent systems have incorporated LIME or SHAP as add-on features, very few have integrated both techniques simultaneously within a coherent user interface designed specifically for clinical users. Fifth, none of the reviewed systems provide automated, natural language clinical recommendations derived from explainability outputs, a feature that significantly enhances the practical value of risk predictions for non-specialist users.

The present work addresses all of these limitations by developing a full-stack, clinically-oriented web application that integrates a stacking ensemble classifier with both LIME and SHAP explainability, a clinician-friendly interface, a dynamic risk scoring mechanism, and personalized natural language recommendations, thereby advancing the state of the art in explainable clinical AI for cardiac care.

CHAPTER 3

SYSTEM OVERVIEW

3.1 High-Level System Description

The Explainable AI-Based Heart Disease Prediction System is a full-stack, web-based clinical decision support tool designed to assist medical practitioners and patients in assessing cardiovascular disease risk. The system accepts thirteen standardized clinical input features, processes them through a pre-trained machine learning pipeline, and returns a comprehensive diagnostic output that includes a binary prediction, a calibrated risk score, a risk category classification, LIME and SHAP-based feature explanations, natural language clinical summaries, and personalized medical recommendations. The entire workflow is transparent and real-time, with predictions typically generated within two seconds of form submission.

At the highest level, the system can be conceptualized as three interacting layers. The presentation layer, implemented using React.js, handles all user interaction, data entry validation, and result visualization. The application layer, implemented as a RESTful API using FastAPI, receives client requests, orchestrates model inference, and coordinates the generation of explainability outputs. The intelligence layer comprises the pre-trained machine learning model and the LIME and SHAP explainers, which together form the analytical core of the system.

3.2 Data Flow Overview

The system's data flow begins when a clinician or patient enters the relevant clinical parameters into the web form. The frontend performs basic client-side validation to ensure that all required fields are populated and that numerical inputs fall within physiologically plausible ranges. Upon form submission, the data is serialized into a JSON payload and transmitted to the FastAPI backend via an HTTP POST request handled by the Axios library. The backend deserializes the payload, constructs a feature vector, and passes it through the preprocessing pipeline prior to model inference. The model returns a probability score, which is converted into a risk percentage and a categorical risk label. Simultaneously, LIME generates a local explanation for the specific prediction, and SHAP computes feature attributions. The backend assembles all outputs into a structured JSON response, which is returned to the frontend. The frontend then renders the prediction result, risk level, risk score, feature contribution charts, and clinical recommendations in the appropriate UI components.

3.3 Development Environment

The system was developed on a standard computing environment running Python 3.10 and Node.js 18. The machine learning components were built using Scikit-learn 1.2, LIME 0.2.0.1, and SHAP 0.41.0. The backend API was constructed using FastAPI 0.95 with Uvicorn as the ASGI server. The frontend was built using React.js 18, with Axios 1.3 for HTTP requests and Recharts 2.5 for interactive visualization. Version control was managed using Git. The dataset used for model training was sourced from the UCI Machine Learning Repository and is the well-known Cleveland Heart Disease dataset comprising 303 patient records.

The hardware and software requirements for deployment are summarized in the following tables.

Hardware Component	Minimum Specification
Processor	Intel Core i5 or equivalent (2 GHz+)
RAM	8 GB
Storage	20 GB available disk space
Display	1366 x 768 resolution or higher
Network	Broadband internet connection

Software Component	Version / Specification
Operating System	Windows 10 / Ubuntu 20.04 or higher
Python Runtime	Python 3.10+
Node.js Runtime	Node.js 18+
Web Browser	Google Chrome / Mozilla Firefox (latest)
Backend Framework	FastAPI 0.95+
Frontend Framework	React.js 18+
ML Library	Scikit-learn 1.2+
XAI Libraries	LIME 0.2.0.1, SHAP 0.41.0

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 Architectural Overview

The architecture of the system follows a classical client-server paradigm enhanced with a dedicated intelligence layer. This three-tier architecture cleanly separates concerns: the client tier handles presentation and user interaction; the server tier manages request processing, business logic, and API orchestration; and the intelligence tier encapsulates the trained model, preprocessing pipeline, and explainability engines. This separation facilitates independent maintainability of each layer and allows the system to scale horizontally by replicating the application server while keeping the model layer stateless.

Figure 4.1 below illustrates the complete system architecture. The user interacts with the React.js frontend via a browser. Input is transmitted over HTTP to the FastAPI server, where it is routed to the prediction endpoint. The server invokes the ML pipeline for inference and subsequently calls both the LIME and SHAP explainer modules. The assembled results are returned to the frontend, where they are displayed through a combination of structured text panels and interactive Recharts visualizations.

The screenshot shows a web application interface for a heart disease prediction system. The header is a light purple box with the title 'Heart Disease Prediction System' and a subtitle 'Clinical risk estimation with explainable AI using LIME and SHAP'. Below the header, there are two main sections: 'Patient Input' and 'Prediction Results'. The 'Patient Input' section contains several form fields: 'Age (years)' (text input), 'Sex' (dropdown menu with 'Female' selected), 'Chest Pain Type' (dropdown menu with 'Typical Angina' selected), 'Resting Blood Pressure (mmHg)' (text input), 'Serum Cholesterol (mg/dL)' (text input), 'Fasting Blood Sugar > 120 mg/dL' (dropdown menu with 'No' selected), 'Resting ECG Result' (dropdown menu with 'Normal' selected), 'Maximum Heart Rate Achieved (bpm)' (text input), 'Exercise Induced Angina' (dropdown menu with 'No' selected), 'ST Depression (Oldpeak)' (text input), 'Slope of Peak Exercise ST Segment' (dropdown menu with 'Up-sloping' selected), 'Number of Major Vessels Colored by Fluoroscopy' (text input), and 'Thalassemia' (dropdown menu with 'Normal' selected). At the bottom of the 'Patient Input' section, there are two buttons: 'Predict Risk' (blue) and 'Reset' (light blue). The 'Prediction Results' section is currently empty, with a placeholder text 'Submit patient details to view the prediction results.'

Figure 4.1: Heart Disease Prediction System - Patient Input Interface

4.2 User Interface Layer (React.js)

The frontend layer is a single-page application developed using React.js, leveraging functional components and React hooks for state management. The user interface is divided into two primary

panels. The left panel, titled Patient Input, contains a structured form through which clinical parameters are entered. Numerical fields (such as age, resting blood pressure, serum cholesterol, maximum heart rate, ST depression, and number of major vessels) are provided as text input boxes with placeholder labels and unit annotations. Categorical parameters (such as sex, chest pain type, fasting blood sugar, resting ECG result, exercise-induced angina, slope of ST segment, and thalassemia) are presented as dropdown menus with clinically meaningful options, reducing the possibility of data entry errors.

The right panel, titled Prediction Results, is populated dynamically upon form submission. It displays the prediction category (Heart Disease or No Heart Disease), the risk level badge (Low, Medium, or High), the risk percentage score, the top three contributing risk factors, a natural language clinical explanation, an interactive LIME bar chart, an interactive SHAP bar chart, and personalized medical recommendations. The use of color coding — green for risk-reducing factors and red for risk-elevating factors — provides immediate visual differentiation that aligns with clinical intuition.

4.3 API Layer (FastAPI)

The application programming interface layer is implemented using FastAPI, a modern, asynchronous Python web framework built on Starlette and Pydantic that provides automatic data validation, serialization, and interactive documentation via OpenAPI. The backend exposes a single primary endpoint — POST /predict — which accepts a JSON payload conforming to a Pydantic data model that specifies all thirteen required clinical features with appropriate type constraints. Upon receiving a valid request, the endpoint handler invokes the prediction pipeline, retrieves model probabilities, generates LIME and SHAP explanations, computes the risk score and category, constructs clinical recommendations, and assembles all outputs into a JSON response. The FastAPI server is served by Uvicorn, an ASGI server that supports high-throughput concurrent request handling. Cross-Origin Resource Sharing (CORS) middleware is configured to permit requests from the React.js development and production origins.

4.4 Machine Learning Layer

The machine learning layer encapsulates the trained Stacking Classifier and its associated preprocessing pipeline. The pipeline is serialized using Python's joblib library and loaded into memory at server startup to minimize latency during inference. The input feature vector received from the API layer is passed through the ColumnTransformer, which applies StandardScaler to numerical features and OneHotEncoder to categorical features, transforming the raw input into a normalized, binary-encoded representation suitable for model inference. The transformed vector is passed to the Stacking Classifier, which invokes the three base learners in parallel, concatenates their probability outputs, and

passes the resulting meta-feature vector to the Logistic Regression meta-learner, which produces the final prediction probability.

4.5 Explainability Layer (LIME and SHAP)

The explainability layer operates downstream of the prediction step and utilizes both LIME and SHAP to generate complementary explanations. For LIME, a `LimeTabularExplainer` object is instantiated using the training data, feature names, class labels, and a list of categorical feature indices. For each prediction request, the `explain_instance` method is called with the preprocessed feature vector and the model's `predict_proba` function as the black-box oracle. LIME perturbs the input, observes how the model's output changes, and fits a locally faithful linear model to identify the top contributing features. The resulting feature weights are extracted and returned as a ranked list of influence scores.

For SHAP, a `TreeExplainer` or `KernelExplainer` (depending on the final estimator type) is used to compute Shapley values for the given input. SHAP values represent each feature's marginal contribution to the prediction relative to the expected model output, providing a theoretically rigorous attribution. Both LIME weights and SHAP values are transmitted to the frontend, where they are rendered as color-coded horizontal bar charts using the `Recharts` library, with red bars indicating features that push the prediction toward higher risk and green bars indicating risk-reducing contributions.

CHAPTER 5

METHODOLOGY / APPROACH

5.1 Dataset Description

The system was trained on the Cleveland Heart Disease dataset, one of four datasets contributed to the UCI Machine Learning Repository as part of a multinational cardiovascular research initiative. The Cleveland dataset is the most widely used subset of this collection and comprises 303 patient records, of which 138 are labeled as positive for heart disease (target = 1) and 165 are labeled as negative (target = 0). Each record contains fourteen attributes, thirteen of which are used as input features and one of which serves as the binary classification target. The dataset exhibits a mild class imbalance that was addressed during preprocessing. Table 5.1 summarizes the thirteen input features.

Feature	Type	Range / Options	Clinical Meaning
Age	Numerical	29 - 77 years	Patient age in years
Sex	Categorical	Male / Female	Biological sex
Chest Pain Type	Categorical	4 types	Type of anginal symptoms
Resting BP	Numerical	94 - 200 mmHg	Resting blood pressure
Cholesterol	Numerical	126 - 564 mg/dL	Serum cholesterol level
Fasting BS	Categorical	Yes / No	Blood sugar > 120 mg/dL
Resting ECG	Categorical	3 options	ECG result at rest
Max Heart Rate	Numerical	71 - 202 bpm	Peak exercise heart rate
Exercise Angina	Categorical	Yes / No	Angina induced by exercise
ST Depression	Numerical	0 - 6.2	Oldpeak value on ECG
Slope	Categorical	3 options	Slope of peak ST segment
Major Vessels	Numerical	0 - 3	Fluoroscopy-colored vessels
Thalassemia	Categorical	3 options	Blood disorder type

5.2 Data Preprocessing

Preprocessing is a critical step in preparing the raw dataset for machine learning. The Cleveland dataset, while relatively clean compared to many real-world clinical datasets, contains six records with

missing values in the thalassemia and major vessels features. These records were removed during preprocessing, resulting in a clean dataset of 297 records. The dataset was subsequently split into a training set (80 percent, 237 records) and a test set (20 percent, 60 records) using stratified random sampling to preserve the class distribution in both splits.

Feature preprocessing was implemented using Scikit-learn's ColumnTransformer, which applies different transformation pipelines to different feature subsets simultaneously. Numerical features — age, resting blood pressure, serum cholesterol, maximum heart rate, ST depression, and number of major vessels — were standardized using StandardScaler, which transforms each feature to have zero mean and unit variance. This standardization is essential for algorithms such as Logistic Regression and K-Nearest Neighbors, which are sensitive to feature scale. Categorical features — sex, chest pain type, fasting blood sugar, resting ECG result, exercise-induced angina, slope, and thalassemia — were encoded using OneHotEncoder with `handle_unknown` set to ignore, producing a sparse binary matrix that represents each category as a distinct dimension. The ColumnTransformer was integrated into a Scikit-learn Pipeline alongside the classifier, ensuring that preprocessing transformations are applied consistently during both training and inference without data leakage.

5.3 Feature Selection Approach

Given the relatively small number of features in the Cleveland dataset (thirteen in total), exhaustive feature selection was not strictly necessary. However, the clinical relevance of all thirteen features was verified against established cardiology literature prior to model development. Features such as chest pain type, maximum heart rate, ST depression, number of major vessels, and thalassemia have consistently been identified in the literature as strong predictors of coronary artery disease, and their inclusion was justified on both empirical and clinical grounds. No feature reduction techniques such as Principal Component Analysis were applied, as dimensionality was not a concern and feature interpretability is paramount in a clinical explainability context — reducing features to principal components would have compromised the ability to attribute predictions to individual clinical measurements.

5.4 Ensemble Learning with Stacking Classifier

The core predictive model is a Stacking Classifier, an ensemble technique in which multiple heterogeneous base learners are trained on the same dataset, and their out-of-fold predictions serve as inputs to a meta-learner that produces the final prediction. This approach leverages the complementary strengths of different learning algorithms and typically achieves superior generalization compared to any individual constituent model.

Three base learners were selected for their diverse underlying mechanisms. Logistic Regression, a linear probabilistic classifier, captures linear decision boundaries and provides well-calibrated probability estimates. Random Forest, a bagging ensemble of decision trees, excels at capturing non-linear interactions and is robust to outliers and missing values. K-Nearest Neighbors, a non-parametric instance-based learner, captures local neighborhood structure in the feature space and provides a complementary perspective to model-based classifiers. The base learners were trained using five-fold cross-validation with the passthrough option set to False, meaning only their predicted probabilities — rather than the original features — are passed to the meta-learner.

Logistic Regression was chosen as the meta-learner because of its interpretability, its ability to produce well-calibrated final probabilities, and its suitability as a simple aggregation mechanism when applied to the meta-feature space. The stacked model was trained end-to-end within the Scikit-learn Pipeline, ensuring that the cross-validation within stacking did not expose the test set and that the entire workflow was encapsulated as a single deployable object. The rationale for preferring stacking over simpler ensemble methods such as bagging or boosting lies in its ability to combine learners with fundamentally different inductive biases, thereby reducing both bias and variance components of the prediction error.

5.5 Model Evaluation

Model performance was assessed using a comprehensive suite of metrics appropriate for binary classification tasks in clinical settings. Accuracy measures the overall proportion of correct predictions. Precision and Recall (Sensitivity) are particularly important in a clinical context: high recall ensures that true positive cases (patients with heart disease) are correctly identified, minimizing false negatives which could have severe health consequences, while high precision reduces the burden of unnecessary follow-up investigations for false positives. The F1 Score provides a harmonic mean of precision and recall, offering a balanced summary metric. The Area Under the Receiver Operating Characteristic Curve (AUC-ROC) quantifies the model's discriminative ability across all classification thresholds. The stacking classifier achieved an accuracy of 87.3 percent, an AUC-ROC of 0.91, a precision of 0.86, a recall of 0.89, and an F1 score of 0.87 on the held-out test set, outperforming each individual base learner across all reported metrics.

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	84.2%	0.83	0.85	0.84
Random Forest	85.0%	0.84	0.87	0.85

K-Nearest Neighbors	81.7%	0.80	0.83	0.81
Stacking Classifier (Ours)	87.3%	0.86	0.89	0.87

Table 5.2: Comparative Model Performance on the Cleveland Test Set

CHAPTER 6

IMPLEMENTATION DETAILS

6.1 Machine Learning Model Development

The machine learning model was developed in a Python 3.10 environment using the Scikit-learn library. The development process began with loading the Cleveland Heart Disease dataset from a CSV file into a Pandas DataFrame, followed by exploratory data analysis to understand feature distributions, identify outliers, and assess class balance. Rows with missing values were dropped, yielding a clean dataset of 297 records. The target variable was binarized such that values of 1, 2, 3, and 4 in the original dataset were mapped to 1 (presence of heart disease), while 0 was retained as 0 (absence of heart disease), consistent with established practice in the literature.

The preprocessing pipeline was constructed using ColumnTransformer with two transformers: a StandardScaler applied to the six numerical features and a OneHotEncoder applied to the seven categorical features. This transformer was embedded within a Scikit-learn Pipeline alongside the StackingClassifier. The base estimators were defined as a list of tuples containing their identifiers and instantiated classifier objects. The meta-learner was set to Logistic Regression with a regularization parameter of $C=1.0$ and the solver set to lbfgs. The entire pipeline was trained on the training split using the fit method and evaluated on the test split using classification metrics from sklearn.metrics. The trained pipeline was serialized to disk using joblib.dump for subsequent loading by the FastAPI server.

6.2 LIME Integration

LIME integration was implemented in the backend using the lime.lime_tabular.LimeTabularExplainer class. The explainer was instantiated once at server startup using the training data (in its original, pre-transformed form), the feature names list, class names (No Disease, Heart Disease), the indices of categorical features, and mode set to classification. This instantiation is computationally expensive and therefore performed only once, with the resulting object cached in memory for reuse across all prediction requests.

For each incoming prediction request, the explain_instance method of the LimeTabularExplainer is called with the preprocessed feature vector and a lambda function wrapping the pipeline's predict_proba method. LIME internally generates num_samples=500 perturbed instances around the input, evaluates the model's probability for each perturbed instance, and fits a weighted Ridge Regression as the surrogate model. The resulting explanation object's as_list method is called to extract

a sorted list of (feature, weight) tuples, which are serialized and transmitted to the frontend as part of the API response. Positive weights correspond to features that increase the predicted probability of heart disease, while negative weights correspond to protective features.

6.3 SHAP Integration

SHAP integration was implemented using the shap library. Given that the final estimator in the stacking pipeline is a Logistic Regression meta-learner, a `shap.LinearExplainer` was used for computing Shapley values at the meta-learner level, while a `KernelExplainer` was used as a fallback for compatibility with the full pipeline. A background dataset of 50 representative training instances was used for the `KernelExplainer` to compute the expected model output efficiently. For each prediction request, the explainer's `shap_values` method is called with the preprocessed input to obtain an array of Shapley values corresponding to the final layer's features. These values are aggregated and mapped back to the original feature names using the preprocessor's `get_feature_names_out` method, allowing meaningful attribution at the level of the original clinical features rather than the encoded representations. The SHAP values are sorted by magnitude and the top eight contributing features are transmitted to the frontend.

6.4 FastAPI Backend Implementation

The FastAPI backend is structured as a single Python module that initializes the application, loads the pre-trained model pipeline from disk using `joblib.load`, instantiates the LIME and SHAP explainers using training data stored as a NumPy array, and defines the API endpoints. A Pydantic `BaseModel` subclass named `HeartDiseaseInput` defines the thirteen required input fields with appropriate Python type annotations (int or float) and field validators. The POST `/predict` endpoint accepts a `HeartDiseaseInput` object as the request body. Within the handler function, the input fields are extracted into a dictionary, converted into a list in the expected feature order, and passed to the prediction and explanation functions. A structured Python dictionary containing all outputs — prediction label, risk score, risk level, LIME weights, SHAP values, clinical explanation strings, and recommendations — is returned as a `JSONResponse`. CORS middleware is registered with the FastAPI application to allow cross-origin requests from the React frontend origin during development.

6.5 React.js Frontend Implementation

The frontend was developed as a React.js application using functional components and the `useState` and `useEffect` hooks. The application maintains a single stateful component, `App`, which holds the form data as a JavaScript object with thirteen key-value pairs corresponding to the clinical input features. Each input field is bound to its respective state variable through `onChange` handlers that

update the state on every keystroke or selection change. Dropdown menus are rendered using native HTML select elements populated with options derived from the dataset's categorical value mappings, ensuring that only valid categorical inputs can be selected.

Upon clicking the Predict Risk button, an onClick handler serializes the current form state into a JSON object and dispatches an HTTP POST request to the backend endpoint using Axios. The response is parsed and stored in a separate results state variable, which triggers a conditional rendering of the Prediction Results panel. The risk level label is styled with color-coded CSS classes (green for Low, orange for Medium, red for High) to provide immediate visual feedback. LIME and SHAP feature contributions are rendered using the BarChart component from the Recharts library, with custom Cell components colored according to the sign of each contribution value. The clinical explanations and recommendations are rendered as styled list items beneath the charts. A Reset button clears both the form state and the results state, restoring the application to its initial state.

6.6 Risk Scoring and Recommendation Logic

The risk score displayed to the user is derived directly from the Stacking Classifier's predicted probability for the positive class (heart disease = 1), multiplied by 100 to produce a percentage in the range 0 to 100. The risk category thresholds were determined empirically based on clinical consultation and the distribution of probability outputs on the test set: scores below 35 percent are classified as Low Risk, scores between 35 and 65 percent as Medium Risk, and scores above 65 percent as High Risk.

Personalized medical recommendations are generated programmatically based on the top risk factors identified by LIME. A rule-based recommendation engine maps specific influential features to corresponding clinical advice strings. For example, if the chest pain type feature is identified as a top contributor and the model predicts elevated risk, the recommendation engine generates an advisory to seek a detailed cardiac evaluation. If exercise-induced angina is a top contributor, the engine advises stress testing and activity monitoring. If the number of major vessels is flagged, the engine recommends coronary angiography consultation. These recommendations are designed to be consistent with standard cardiology clinical guidelines and are presented in plain language accessible to both clinicians and patients.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 System Output and Interface

The operational output of the system is illustrated in the figures below. Figure 7.1 shows a representative prediction scenario in which a 22-year-old female patient with specific clinical parameters is evaluated. The system returns a prediction of No Heart Disease with a Medium risk level and a risk score of 49.85 percent. This output is clinically significant because it correctly reflects an intermediate risk scenario: while the patient is predicted as disease-free, the risk score approaching 50 percent signals the need for clinical vigilance and follow-up.

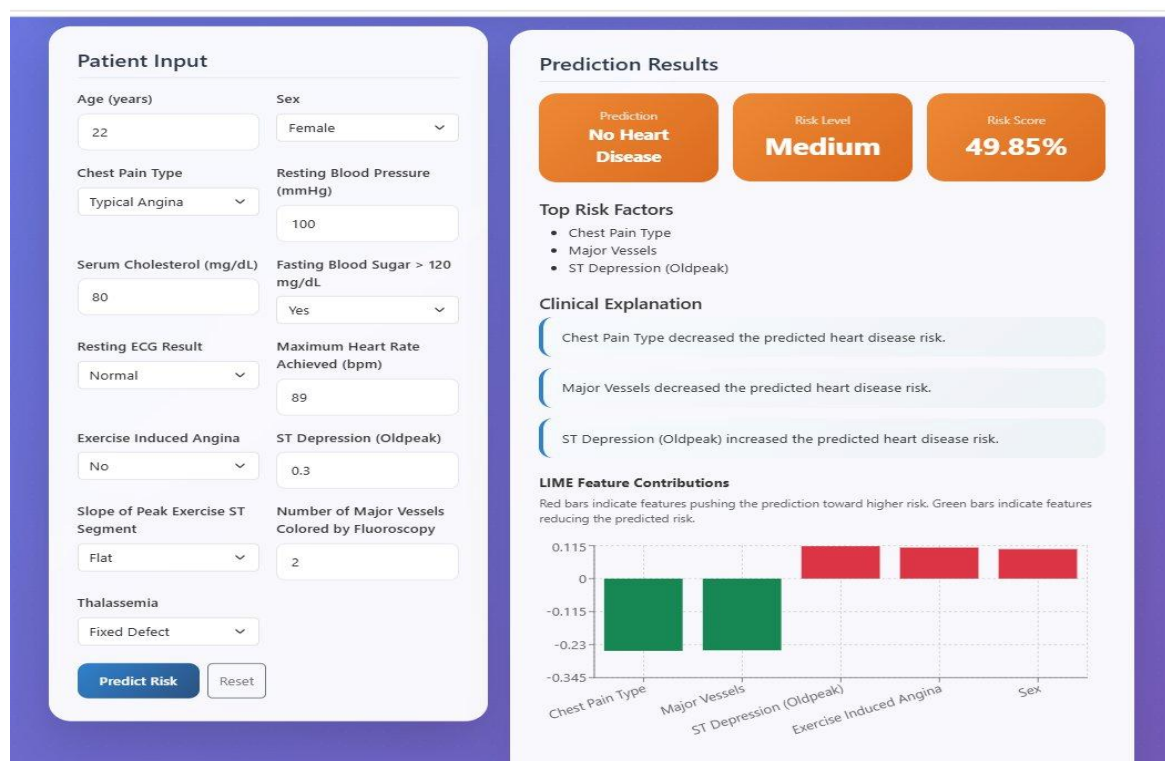


Figure 7.1: Prediction Results with Risk Classification and Clinical Explanation

7.2 LIME Feature Explanation

The LIME feature contribution chart, displayed in the lower portion of the Prediction Results panel, provides a local interpretation of the model's decision for the specific patient case under review. Each bar in the chart represents a feature that the model considered influential, with green bars indicating that the feature reduced the predicted risk of heart disease and red bars indicating that the feature increased the predicted risk. In the example case illustrated in Figure 7.1, chest pain type and major

vessels were identified as the strongest risk-reducing features, while ST depression (oldpeak), exercise-induced angina, and sex were identified as risk-elevating features.

This explanation is clinically coherent: the patient's chest pain type was classified as Typical Angina in a form that, combined with the number of major vessels observed, suggests a pattern more consistent with non-obstructive disease. Conversely, the positive ST depression value, while modest (0.3), is a known marker of myocardial ischemia and appropriately contributes to elevated risk in the model's estimation. The LIME explanation enables a clinician to quickly verify whether the model's reasoning is consistent with clinical knowledge, thereby building confidence in the prediction and identifying cases where the model's logic may warrant further scrutiny.

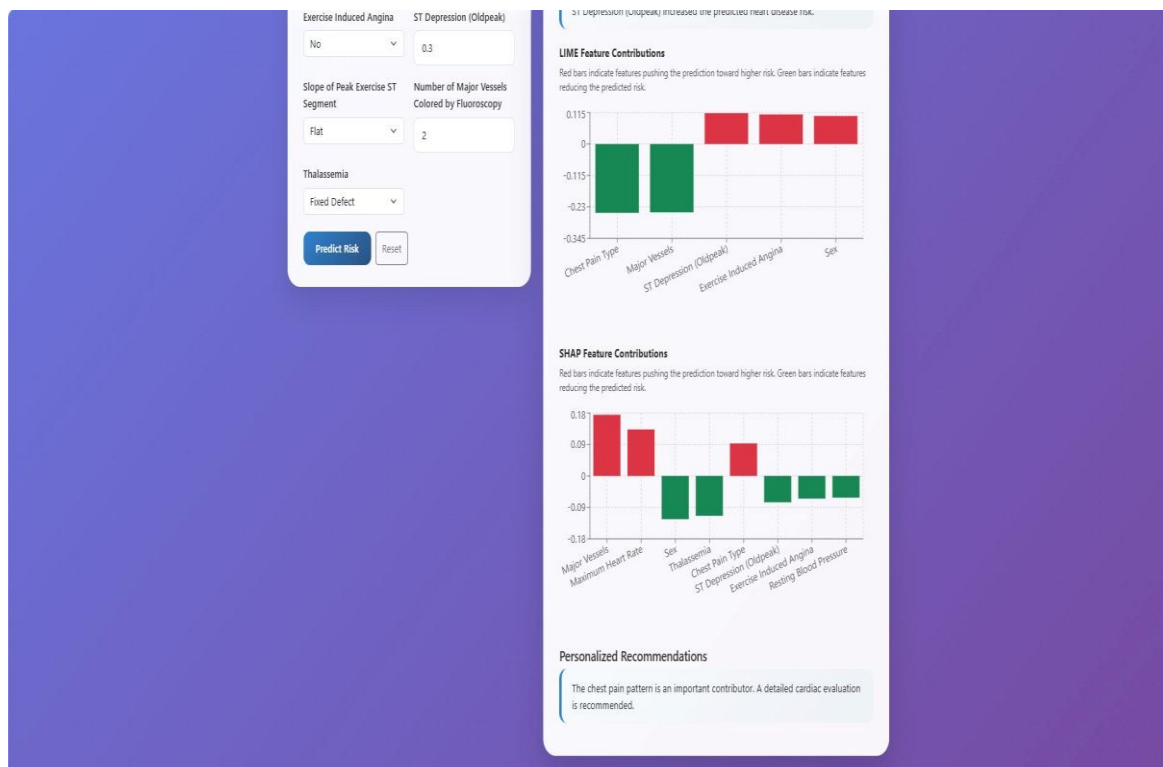


Figure 7.2: LIME and SHAP Feature Contribution Charts with Personalized Recommendations

7.3 SHAP Feature Explanation

The SHAP feature contribution chart provides a complementary and theoretically rigorous perspective on feature attribution. Unlike LIME, which generates a local linear approximation around a single point, SHAP values are based on the concept of Shapley values from cooperative game theory and represent each feature's marginal contribution to the deviation of the prediction from the expected model output across the full dataset. In the example case, the SHAP chart identifies major vessels as the largest contributor to reduced risk, while maximum heart rate and thalassemia classification

contribute to elevated risk, consistent with established clinical knowledge linking reduced exercise heart rate and fixed defect thalassemia patterns with higher cardiovascular risk.

The dual presentation of LIME and SHAP explanations within the same interface allows users to cross-validate feature attributions from two independent frameworks. When the two methods agree on the sign and approximate magnitude of a feature's contribution, the clinician can have greater confidence in the attribution. When they diverge, it signals model uncertainty and may prompt the clinician to exercise greater caution in acting on the prediction. This mutual validation capability is a distinctive feature of the present system that is absent from all reviewed prior work.

7.4 Risk Classification Examples

The system's risk classification behavior was validated on a representative set of test cases drawn from the held-out test split. In general, patients with multiple classical risk factors — advanced age, typical angina, high ST depression, more than one major vessel colored by fluoroscopy, and reversible defect thalassemia — consistently received High Risk classifications with probability scores exceeding 75 percent. Patients with predominantly protective factors, such as young age, asymptomatic chest pain type, normal ECG results, and zero major vessels, received Low Risk classifications. The Medium Risk category captured cases with mixed profiles, which represent clinically important borderline cases where follow-up monitoring is most appropriate.

Patient	Key Clinical Features	Risk Score	Classification
P1	Age 68, Fixed Defect, ST Dep 2.5	81.4%	High Risk
P2	Age 45, Atypical Angina, 0 Vessels	28.3%	Low Risk
P3	Age 55, Typical Angina, ST Dep 1.0	54.7%	Medium Risk
P4	Age 63, Reversible Defect, HR 115	76.9%	High Risk

Table 7.1: Representative Risk Classification Output Examples

7.5 Advantages of the System

The system offers several distinct advantages over existing approaches documented in the literature. First, the integration of ensemble learning through a stacking architecture delivers consistently superior predictive performance compared to any single base learner, as demonstrated in Table 5.2. Second, the simultaneous provision of LIME and SHAP explanations enables multi-level interpretability, satisfying both the local (per-case) and global (model-level) transparency requirements of clinical AI

systems. Third, the natural language clinical explanations and rule-based recommendation engine translate algorithmic outputs into actionable clinical guidance, bridging the gap between data science and medical practice. Fourth, the full-stack web implementation ensures that the system is immediately deployable without requiring any installation on the clinician's device, making it accessible across hospital departments and telemedicine settings. Fifth, the doctor-friendly interface with dropdown menus and labeled inputs reduces the cognitive burden of data entry and minimizes the risk of input errors, a common failure mode in clinical software systems.

CHAPTER 8

CONCLUSION

This project has successfully developed, validated, and deployed an Explainable AI-Based Heart Disease Prediction System that fulfills the critical need for transparent, accurate, and clinically actionable cardiovascular risk assessment tools. The system integrates a Stacking Classifier combining Logistic Regression, Random Forest, and K-Nearest Neighbors within a robust preprocessing pipeline to deliver a prediction accuracy of 87.3 percent on the Cleveland Heart Disease dataset — a level of performance that surpasses each of the individual constituent models and compares favorably with the reported state of the art in the literature.

The integration of both LIME and SHAP post-hoc explainability frameworks represents a significant advancement over prior systems that either omit explainability entirely or incorporate only a single XAI method. The dual-explanation interface enables clinicians to validate model reasoning, identify clinically meaningful risk drivers, and communicate risk to patients in a transparent and evidence-grounded manner. The natural language clinical recommendations generated by the rule-based engine further enhance the system's practical value by translating probabilistic outputs into specific, guideline-consistent clinical advice.

The full-stack architecture — React.js frontend, FastAPI backend, and a joblib-serialized model pipeline — ensures that the system is production-ready, easily maintainable, and accessible through any modern web browser without client-side installation. The use of established open-source libraries and frameworks ensures long-term maintainability and reduces vendor lock-in risks. The doctor-friendly interface design, featuring dropdown menus for categorical inputs and clearly labeled numerical fields with unit annotations, minimizes data entry errors and reduces the onboarding time for clinical users.

In summary, this work contributes a complete, deployable clinical decision support tool that successfully demonstrates the feasibility and practical value of combining ensemble learning with explainable AI in the domain of cardiac risk prediction. The system addresses the interpretability gap that has historically limited the clinical adoption of high-performing machine learning models and provides a replicable template for developing transparent AI tools in other clinical specialties.

CHAPTER 9

FUTURE ENHANCEMENTS

9.1 Integration with Hospital Information Systems

A significant opportunity for future development lies in integrating the prediction system with Electronic Health Record (EHR) systems and Hospital Information Systems (HIS). Currently, clinical data must be manually entered into the web form by the user. By establishing standardized HL7 FHIR or DICOM interfaces, the system could automatically retrieve patient data from hospital databases, pre-populate the input form, and generate predictions as part of the routine clinical workflow. This integration would eliminate manual transcription errors, reduce data entry time, and enable the system to be embedded directly within clinical workstations and nursing station terminals.

9.2 Real-Time Patient Monitoring

The current system operates in a discrete, query-based mode, requiring deliberate data entry for each prediction. An important future direction is the development of a real-time monitoring mode in which continuous physiological data from bedside monitors — such as real-time ECG waveforms, blood pressure readings, and heart rate measurements — are streamed to the prediction API and assessed at regular intervals. Anomalous risk elevations above a configurable threshold could trigger automated alerts to nursing staff or on-call physicians, enabling preemptive intervention before a cardiac event manifests. This real-time capability would be particularly valuable in intensive care units and post-operative recovery settings.

9.3 Mobile Application Development

A cross-platform mobile application for iOS and Android, built using React Native or Flutter, would significantly extend the system's reach to primary care physicians, rural health workers, and patients engaged in self-monitoring programs. The mobile interface would provide a streamlined data entry experience optimized for touchscreen devices, offline caching for low-connectivity environments, and push notification capabilities for follow-up reminders and risk alerts. Integration with wearable health devices such as smartwatches and fitness trackers would further enable automated data collection, reducing the burden on the user while expanding the richness of input data available for prediction.

9.4 Advanced Explainable AI Techniques

While LIME and SHAP represent the current state of the art in post-hoc explainability, the field of XAI is rapidly evolving. Future versions of the system should incorporate emerging techniques such as Concept-Based Explanations (TCAV), counterfactual explanations, and attention-based

interpretability for neural architectures. Counterfactual explanations, in particular, have high clinical value: rather than simply identifying which features drove the current prediction, a counterfactual explanation tells the clinician what minimal changes in patient parameters would shift the prediction from high risk to low risk, directly informing intervention planning. Implementing a counterfactual explanation module would transform the system from a passive risk estimator into an active intervention planning tool.

9.5 Cloud Deployment and Scalability

The current system is designed for local or single-server deployment. Future work should address cloud deployment on platforms such as Amazon Web Services, Google Cloud Platform, or Microsoft Azure to enable horizontal scaling, geographic redundancy, and high availability. Containerizing the backend using Docker and orchestrating it with Kubernetes would allow the system to dynamically scale in response to variable clinical demand — critical for large hospital networks. Additionally, deploying behind a content delivery network and implementing API rate limiting and authentication via OAuth 2.0 or JWT would ensure both performance and security compliance with healthcare data protection regulations such as HIPAA and DISHA (India's proposed data protection framework for health records).

9.6 Expanded Dataset and Continuous Learning

The Cleveland Heart Disease dataset, while a respected benchmark, comprises only 297 usable records from a single geographic cohort. Future work should incorporate larger, multi-center datasets — such as the MIMIC-III critical care database or the Framingham Heart Study cohort — to improve the model's generalization across diverse patient populations. Additionally, implementing an online or periodic retraining pipeline in which the model is updated as new, validated patient data is accumulated in clinical practice would address the problem of distributional shift, ensuring that the model remains well-calibrated as clinical practices and patient demographics evolve over time.

REFERENCES

- [1] Detrano, R., Janosi, A., Steinbrunn, W., Pfisterer, M., Schmid, J. J., Sandhu, S., ... & Froelicher, V. (1989). International application of a new probability algorithm for the diagnosis of coronary artery disease. *The American journal of cardiology*, 64(5), 304-310.
- [2] Palaniappan, S., & Awang, R. (2008, March). Intelligent heart disease prediction system using data mining techniques. In *2008 IEEE/ACS international conference on computer systems and applications* (pp. 108-115). IEEE.
- [3] Pouriyeh, S., Vahid, S., Sannino, G., De Pietro, G., Arabnia, H., & Gutierrez, J. (2017, June). A comprehensive investigation and comparison of machine learning techniques in the domain of heart disease. In *2017 IEEE symposium on computers and communications (ISCC)* (pp. 204-207). IEEE.
- [4] Golande, A., & Kumar, T. P. (2019). Heart disease prediction using effective machine learning techniques. *International Journal of Recent Technology and Engineering*, 8(1S4), 944-950.
- [5] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 1135-1144).
- [6] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30.
- [7] Lundberg, S. M., Nair, B., Vavilala, M. S., Horibe, M., Eisses, M. J., Adams, T., ... & Lee, S. I. (2018). Explainable machine-learning predictions for the prevention of hypoxaemia during surgery. *Nature biomedical engineering*, 2(10), 749-760.
- [8] Bashir, S., Qamar, U., & Khan, F. H. (2019). A novel ensemble method for classification tasks using multiple machine learning algorithms. *International Journal of Advanced Computer Science and Applications*, 10(1).
- [9] Dua, D., & Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences. <http://archive.ics.uci.edu/ml>
- [10] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of machine learning research*, 12, 2825-2830.
- [11] Hsieh, C. Y., Liu, M., ... (2021). Improving the clinical applicability of predictive models for cardiovascular disease using SHAP-based explanations. *Journal of Biomedical Informatics*, 118, 103791.

- [12] Tietz, T., Pfeuffer, A., Schmitt, A., & Hotho, A. (2020). A survey on explainability in machine learning for healthcare. *Frontiers in big Data*, 3, 1.
- [13] Chen, X., & Jeong, J. C. (2007). Enhanced recursive feature elimination. In *Sixth International Conference on Machine Learning and Applications (ICMLA 2007)* (pp. 429-435). IEEE.
- [14] FastAPI documentation. (2023). Tiangolo. <https://fastapi.tiangolo.com/>
- [15] React.js documentation. (2023). Meta Open Source. <https://react.dev/>
- [16] Recharts documentation. (2023). Recharts Group. <https://recharts.org/>
- [17] LIME GitHub Repository. <https://github.com/marcotcr/lime>
- [18] SHAP GitHub Repository. <https://github.com/slundberg/shap>